MINI PROJECT REPORT ON

"DevOps Monitoring System Using Docker, Jenkins, Prometheus, Grafana, and Nagios"

SUBMITTED IN PARTIAL FULFILLMENT OF THE REQUIREMENTS OF DEGREE OF

BACHELOR OF ENGINEERING BY

Rohan Thakur (T23, 2401121)

Jiya Jagiasi (T23, 2511129)

Siddhi Lahoti (T23, 2511132)

SUBJECT

Agile Software Development and DevOps

SUPERVISOR

Prof. Sanober Shaikh



DEPARTMENT OF ARTIFICIAL INTELLIGENCE & DATA SCIENCE
THADOMAL SHAHANI ENGINEERING COLLEGE, ADVOCATE NARI GURSAHANI MARG, 37TH RD, OFF LINKING ROAD, TPS III, BANDRA (WEST), MUMBAI, 400050
UNIVERSITY OF MUMBAI
[Academic Year: 2026–27]

THADOMAL SHAHANI ENGINEERING COLLEGE, ADVOCATE NARI GURSAHANI MARG, 37TH RD, OFF LINKING ROAD, TPS III, BANDRA (WEST), MUMBAI, 400050 A.Y. - 2026-27

Certificate

This is to certify that the Mini Project entitled **"DevOps Monitoring System Using Docker, Jenkins, Prometheus, Grafana, and Nagios"** is a bonafide work of **Rohan Thakur, Jiya Jagiasi, Siddhi Lahoti** submitted to the University of Mumbai in partial fulfilment of the requirement for the award of the degree of **"Undergraduate"** in **"Artificial Intelligence and Data Science".**

Prof. Sanober Shaikh
(Supervisor)

Abstract

The DevOps Monitoring System is a containerised monitoring and delivery setup that brings application health checking, metrics collection, visualisation and host monitoring into one place. A web dashboard shows the live status of the application together with CPU usage, memory usage and uptime, and provides quick links to the Prometheus, Grafana and Nagios interfaces. The monitoring services are packaged with Docker and started together using Docker Compose, which removes the need to install and configure each tool by hand.

Beyond the dashboard, the project demonstrates a complete DevOps toolchain. Prometheus scrapes metrics from its own endpoint and from cAdvisor, Grafana visualises the collected data in a DevOps Monitoring dashboard, and Nagios checks the status of the host through ping-based checks. A Jenkins pipeline named DevOps-Monitor-Pipeline automates the Checkout, Build, Test, Build Image and Deploy stages, so that every change to the project is built and verified in a repeatable way.

TABLE OF CONTENTS

Content **Page No.**

Abstract **3**

List of Figures **5**

1. Introduction **6**

1.1 Background 6

1.2 Problem Statement 6

2. Implementation **7**

2.1 Technologies Used 7

2.2 System Architecture 7

2.3 Service and Port Configuration 8

2.4 CI/CD Pipeline Design 8

2.5 Monitoring Workflow 8

3. Results and Discussion **9**

3.1 Result 9

3.2 Discussion 12

4. Conclusion **14**

5. References **15**

LIST OF FIGURES

Fig. 3.1 DevOps Monitoring Dashboard (Application).....10
Fig. 3.2 Docker Containers Running (Monitoring Stack).....10
Fig. 3.3 Grafana Dashboard (Prometheus Target Status).....11
Fig. 3.4 Prometheus Targets (Metrics Collection).....11
Fig. 3.5 Nagios Host Status (Host Monitoring).....12
Fig. 3.6 Jenkins Pipeline (CI/CD).....12

1. Introduction

1.1 Background

Modern applications are expected to be available around the clock, but deploying them and keeping an eye on their health is still done manually in many small projects. A developer builds the application by hand, copies it to a server, and then logs in to check whether it is running, how much CPU and memory it is using and whether any container has stopped. When something fails, the problem is often noticed only after users complain, which increases downtime.

DevOps practices address this by automating the path from code to running service and by making the running system observable. Containers make the application and its dependencies portable, a CI/CD server such as Jenkins repeats the build and deployment steps on every change, and monitoring tools collect and display the health of the system continuously. Prometheus and Grafana provide time-series metrics and dashboards, while Nagios adds host and service status checks.

This project, the DevOps Monitoring System, combines these tools in one setup. It uses Docker and Docker Compose to run the services, Jenkins for CI/CD automation, Prometheus with cAdvisor for metrics collection, Grafana for visualisation and Nagios for host monitoring. A web dashboard ties them together by showing application status and resource usage and by linking to each monitoring tool.

1.2 Problem Statement

Traditional application deployment and monitoring often require manual effort to check application health, manage containers, and identify system failures. The lack of centralized monitoring and automated deployment can delay problem detection and increase downtime.

The proposed DevOps Monitoring System integrates Docker for containerization, Jenkins for CI/CD automation, Prometheus for metrics collection, Grafana for visualization, and Nagios for host and service monitoring. The system aims to simplify deployment, improve visibility into system health, and help identify issues more efficiently through monitoring and visualization tools.

The system therefore aims to provide the following capabilities:

1. Containerise the application and the monitoring services with Docker and start them together using Docker Compose.
2. Automate the build, test, image creation and deployment steps through a Jenkins CI/CD pipeline.
3. Collect application and container metrics centrally using Prometheus and cAdvisor.
4. Visualise the collected metrics on a Grafana dashboard so that system health can be seen at a glance.
5. Monitor the host and its services with Nagios and report their status as Up, Down, Unreachable or Pending.
6. Provide a single web dashboard that shows application status, CPU usage, memory usage and uptime, with quick links to Prometheus, Grafana and Nagios.

Chapter 2
Implementation

2.1 Technologies Used

- **Application dashboard** – A web dashboard served on port 8501 shows the application status, CPU usage, memory usage and uptime. Its sidebar has Dashboard, System Health, Logs and About pages, and its Quick Links section opens Prometheus, Grafana and Nagios.
- **Containerisation** – Docker packages the services as containers, and Docker Compose (monitoring/compose.yaml) defines and starts the monitoring stack, including the devops-prometheus and devops-grafana containers, with a single command.
- **CI/CD automation** – Jenkins (port 8080) runs the pipeline DevOps-Monitor-Pipeline and reports every build with its stages and their durations.
- **Metrics collection** – Prometheus (port 9090) scrapes the configured targets and stores the results as time-series data.
- **Container metrics** – cAdvisor exposes the CPU, memory and network usage of running containers on port 8080 for Prometheus to scrape.
- **Visualisation** – Grafana (port 3000) uses Prometheus as its data source and presents the metrics in the DevOps Monitoring dashboard.
- **Host and service monitoring** – Nagios (port 8085) tracks the status of hosts, shows the number of hosts that are Up, Down, Unreachable or Pending, and displays the status information of each check.
- **Version control** – Git, from which the Checkout stage of the Jenkins pipeline fetches the project source.

2.2 System Architecture

The system is organised as a set of cooperating services. Each service runs on its own port and has a single role in the delivery and monitoring flow.

- **Dashboard application** – the entry point for users on port 8501. It summarises application health and links out to the monitoring tools.
- **monitoring/compose.yaml** – the Docker Compose file that defines the monitoring stack. The docker compose ps command is used with it to confirm that the containers are running.
- **Prometheus and cAdvisor** – Prometheus collects metrics from itself (prometheus:9090) and from cAdvisor (cadvisor:8080), which reports container-level resource usage.
- **Grafana** – reads from the Prometheus data source and shows panels such as Prometheus Target Status.
- **Nagios** – checks the localhost host with a ping check and shows the result on its Host Status Details page.
- **Jenkins** – checks out the source, builds, tests, builds the Docker image and deploys, as shown by the stages of the pipeline.

In summary, containers expose metrics, Prometheus collects them, Grafana displays them, Nagios checks host availability and Jenkins keeps delivery repeatable.

2.3 Service and Port Configuration

Each tool in the stack runs on the local machine and is reached through localhost and its own port. Table 2.1 lists the services, the ports observed in the project and the role each one plays.

Service	Port	Role
Dashboard application	8501	Application status, CPU, memory and uptime summary
Jenkins	8080	CI/CD pipeline (DevOps-Monitor-Pipeline)
Prometheus	9090	Metrics collection; targets page at /targets
Grafana	3000	Dashboards and visualisation
cAdvisor	8080	Container resource metrics (scraped as cadvisor:8080)
Nagios	8085	Host and service status monitoring

Table 2.1: Services and ports in the DevOps Monitoring System

2.4 CI/CD Pipeline Design

The Jenkins pipeline DevOps-Monitor-Pipeline automates the path from source code to a running container. It is organised into five stages that run in order:

1. **Checkout** – the project source is fetched from the repository (2 s).
2. **Build** – the application is built (18 s).
3. **Test** – the checks defined for the project are run (12 s).
4. **Build Image** – the Docker image is built (25 s).
5. **Deploy** – the container is deployed (15 s).

If a stage fails, the pipeline stops and the build is marked as failed, so a broken change does not reach deployment. The durations above were recorded for build #3, which finished with the status Success after about 72 seconds in total.

2.5 Monitoring Workflow

Prometheus is configured with scrape targets for itself and for cAdvisor. Each target exposes a /metrics endpoint, which Prometheus reads and stores. The Targets page of Prometheus reports the state of every target together with the time of its last scrape and the scrape duration, as listed in Table 2.2.

Target	Endpoint	State	Last scrape	Scrape duration
prometheus	http://prometheus:9090/metrics	UP	7.5s ago	52ms
cadvisor	http://cadvisor:8080/metrics	UP	8.2s ago	48ms

Table 2.2: Prometheus targets observed in the project

Grafana is connected to Prometheus as a data source. The DevOps Monitoring dashboard contains a Prometheus Target Status panel that plots the up query, which returns 1 for a reachable target and 0 for an unreachable one.

Nagios works independently of Prometheus. It checks the localhost host with a ping check and reports the result as Up, Down, Unreachable or Pending, together with the time of the last check, its duration and status information such as packet loss and round-trip time.

Chapter 3
Results and Discussion

3.1 Result

- **Application dashboard:** the DevOps Monitoring Dashboard at localhost:8501 shows Application Status as Running, CPU Usage of 12.5%, Memory Usage of 48.3% and an Uptime of 2d 4h 32m, with Quick Links to Prometheus, Grafana and Nagios.
- **Docker containers:** the command docker compose -f monitoring/compose.yaml ps lists devops-grafana (grafana/grafana:latest) and devops-prometheus (prom/prometheus:latest) as up, with ports 3000 and 9090 published.
- **Grafana:** the DevOps Monitoring dashboard uses Prometheus as its data source and plots the up query in the Prometheus Target Status panel, where the value stays at 1.0.
- **Prometheus:** the Targets page shows both prometheus and cadvisor in the UP state, with scrape durations of 52 ms and 48 ms.
- **Nagios:** the Host Status Details page shows 1 host Up and 0 Down, Unreachable or Pending. The localhost host is UP with the status information PING OK, packet loss 0% and RTA 0.21 ms.
- **Jenkins:** the DevOps-Monitor-Pipeline shows build #3 as Success, with Checkout (2s), Build (18s), Test (12s), Build Image (25s) and Deploy (15s) all completed.

Summary of observed results

To check the whole stack, the status of each tool was recorded from its own interface. Table 3.1 summarises the observations.

Tool	Address	Observation	Status
Dashboard app	localhost:8501	Application running; CPU 12.5%, memory 48.3%, uptime 2d 4h 32m	Running
Docker Compose	monitoring/ compose.yaml	devops-grafana and devops-prometheus containers up for 2 minutes	Up
Prometheus	localhost:9090	prometheus and cadvisor targets scraped in 52 ms and 48 ms	UP
Grafana	localhost:3000	Prometheus Target Status panel plots up at 1.0	Working
Nagios	localhost:8085	localhost host: PING OK, packet loss 0%, RTA 0.21 ms	UP
Jenkins	localhost:8080	Build #3 of DevOps-Monitor-Pipeline completed all five stages	Success

Table 3.1: Observed status of each tool in the DevOps Monitoring System

All six components reported a healthy state at the time of the screenshots. Values such as CPU and memory usage are single readings rather than averages, a limitation discussed in Section 3.2.

Screenshots

The following screenshots show the working of the DevOps Monitoring System and the tools integrated in it.



Fig. 3.1: DevOps Monitoring Dashboard (Application)

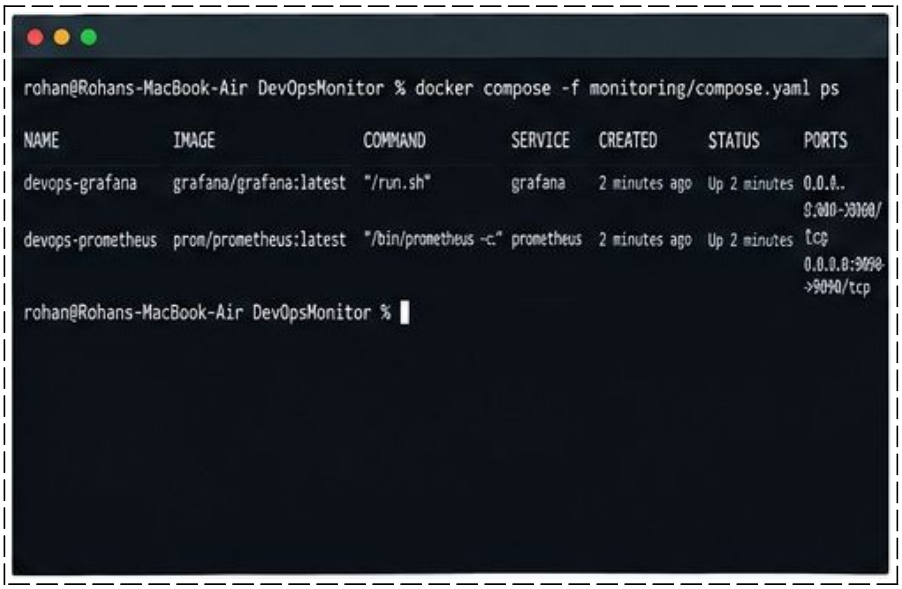


Fig. 3.2: Docker Containers Running (Monitoring Stack)



Fig. 3.3: Grafana Dashboard (Prometheus Target Status)

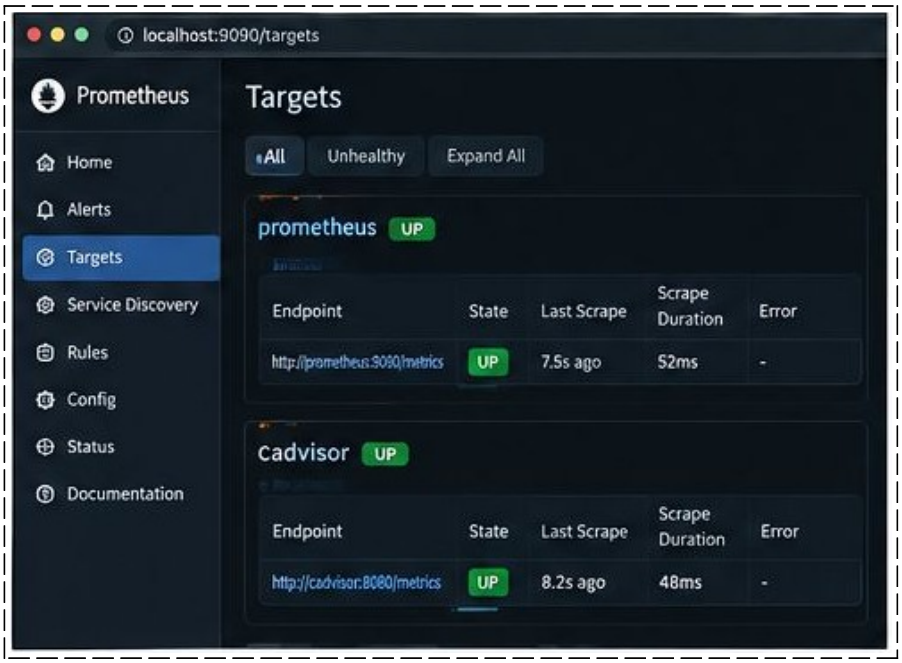


Fig. 3.4: Prometheus Targets (Metrics Collection)

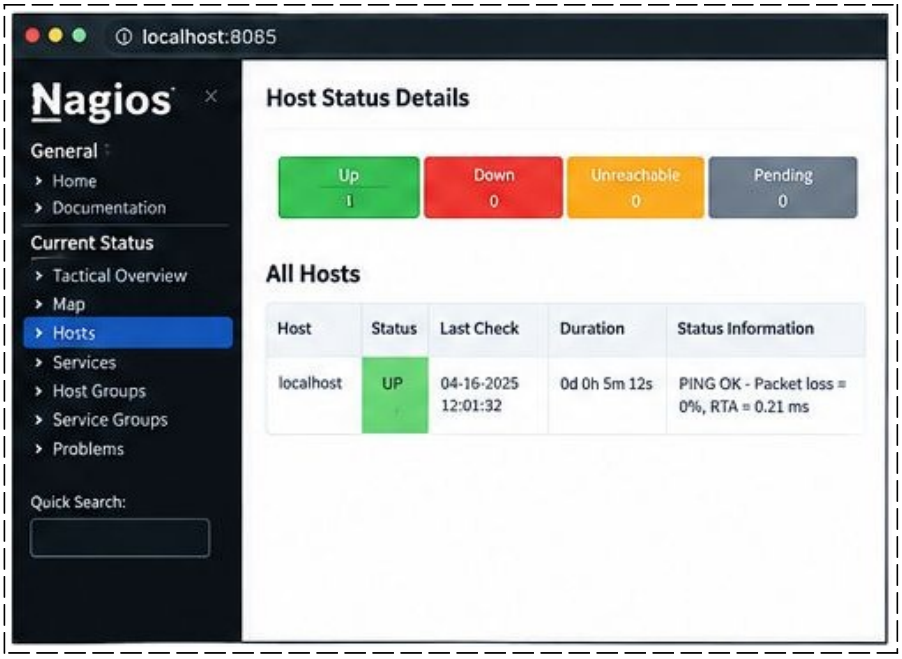


Fig. 3.5: Nagios Host Status (Host Monitoring)

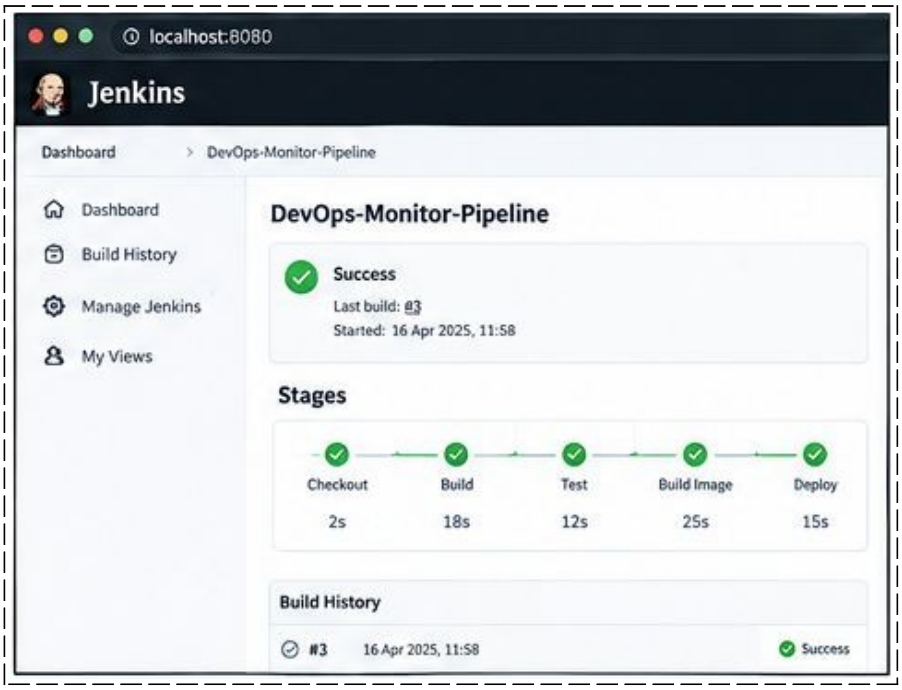


Fig. 3.6: Jenkins Pipeline (CI/CD)

3.2 Discussion

The DevOps Monitoring System shows that a complete delivery and monitoring workflow can be assembled from open-source tools running on a single machine. Docker Compose lets the monitoring stack be started with one command, the Jenkins pipeline repeats the build steps on every change, and the dashboard gives a quick view of application health, so the manual checks described in the problem statement are replaced by automated ones.

The tools also complement each other rather than overlap. Prometheus and Grafana provide time-based metrics and visual dashboards, cAdvisor adds per-container resource usage, and

Nagios gives a separate availability view of the host through ping checks. Using both a metrics-based and a check-based tool means that a problem can be seen from two directions.

The following limitations were identified while reviewing the project, and they define what would be needed before real-world use:

- **Limited dashboard content.** The Grafana dashboard shows a single Prometheus Target Status panel based on the up query. Panels for CPU, memory and network usage from cAdvisor, and for application-specific metrics, would make it more useful.
- **Small monitoring scope.** Nagios monitors only the localhost host and Prometheus has two targets. Monitoring several hosts, services and ports would need additional configuration. No alert rules or notifications are shown in the screenshots.
- **Single-host deployment.** Docker Compose suits a single machine. High availability, scaling and rolling updates would need an orchestrator such as Kubernetes.
- **Pipeline scope.** The Jenkins pipeline runs on a local Jenkins server. The strength of the Test stage depends on the checks defined in it, and deployment targets the local environment rather than a remote server.
- **Snapshot values.** Figures such as 12.5% CPU and 48.3% memory are single readings, so they say little about behaviour over time or under load.

No throughput, latency or load-testing figures are claimed in this report, because no performance benchmarking has been carried out; the values in the screenshots are single observations of a working system.

Chapter 4
Conclusion

4.1 Conclusion

The DevOps Monitoring System brings deployment automation and monitoring of an application into one setup. Docker and Docker Compose package and start the services, Jenkins automates the pipeline from source to deployment, Prometheus and cAdvisor collect application and container metrics, Grafana visualises them, and Nagios reports the status of the host. A web dashboard summarises application health and links to each tool. Together they reduce manual effort, shorten the time taken to notice a failure, and make the system easier to reproduce.

4.2 Future Scope

- Add Grafana panels for CPU, memory, network and application metrics, and provision the dashboards as code.
- Configure alerting with Prometheus Alertmanager and Nagios notifications by email or chat.
- Extend Prometheus and Nagios to monitor multiple hosts, services and ports.
- Strengthen the Jenkins pipeline with automated tests, image scanning and deployment to a remote server.
- Move from Docker Compose to Kubernetes for scaling, and add log aggregation to support the Logs page of the dashboard.

Chapter 5
References

1. Docker Documentation – https://docs.docker.com/
2. Docker Compose Documentation – https://docs.docker.com/compose/
3. Jenkins Pipeline Documentation – https://www.jenkins.io/doc/book/pipeline/
4. Prometheus Documentation – https://prometheus.io/docs/
5. Grafana Documentation – https://grafana.com/docs/
6. Nagios Documentation – https://www.nagios.org/documentation/
7. cAdvisor – https://github.com/google/cadvisor
8. Git Documentation – https://git-scm.com/doc